

2798. Number of Employees Who Met the Target

Solved

Easy

Topics

Companies

Hint

There are `n` employees in a company, numbered from `0` to `n - 1`. Each employee `i` has worked for `hours[i]` hours in the company.

The company requires each employee to work for **at least** `target` hours.

You are given a **0-indexed** array of non-negative integers `hours` of length `n` and a non-negative integer `target`.

Return *the integer denoting the number of employees who worked at least* `target` *hours*.

Example 1:

Input: `hours = [0,1,2,3,4]`, `target = 2`

Output: `3`

Explanation: The company wants each employee to work for at least 2 hours.

- Employee 0 worked for 0 hours and didn't meet the target.
- Employee 1 worked for 1 hours and didn't meet the target.
- Employee 2 worked for 2 hours and met the target.
- Employee 3 worked for 3 hours and met the target.
- Employee 4 worked for 4 hours and met the target.

There are 3 employees who met the target.

Example 2:

Input: `hours = [5,1,4,2,2]`, `target = 6`

Output: `0`

Code

Java Auto

```
1 class Solution {
2     public int numberOfEmployeesWhoMetTarget(int[] hours, int target) {
3         int count = 0;
4
5         for (int h : hours) {
6             if (h >= target) {
7                 count++;
8             }
9         }
10
11         return count;
12     }
}
```

Saved

Ln 13, Col 2

Testcase Test Result

Accepted Runtime: 0 ms

Case 1

Case 2

Input

hours =
[0,1,2,3,4]

target =
2

Output

3

2824. Count Pairs Whose Sum is Less than Target

Solved

Easy

Topics

Companies

Hint

Given a **0-indexed** integer array `nums` of length `n` and an integer `target`, return the number of pairs `(i, j)` where `0 <= i < j < n` and `nums[i] + nums[j] < target`.

Example 1:

Input: `nums = [-1,1,2,3,1]`, `target = 2`
Output: `3`
Explanation: There are 3 pairs of indices that satisfy the conditions in the statement:
- `(0, 1)` since `0 < 1` and `nums[0] + nums[1] = 0 < target`
- `(0, 2)` since `0 < 2` and `nums[0] + nums[2] = 1 < target`
- `(0, 4)` since `0 < 4` and `nums[0] + nums[4] = 0 < target`
Note that `(0, 3)` is not counted since `nums[0] + nums[3]` is not strictly less than the target.

Example 2:

Input: `nums = [-6,2,5,-2,-7,-1,3]`, `target = -2`
Output: `10`
Explanation: There are 10 pairs of indices that satisfy the conditions in the statement:
- `(0, 1)` since `0 < 1` and `nums[0] + nums[1] = -4 < target`
- `(0, 3)` since `0 < 3` and `nums[0] + nums[3] = -8 < target`
- `(0, 4)` since `0 < 4` and `nums[0] + nums[4] = -13 < target`
- `(0, 5)` since `0 < 5` and `nums[0] + nums[5] = -7 < target`
- `(0, 6)` since `0 < 6` and `nums[0] + nums[6] = -3 < target`
- `(1, 4)` since `1 < 4` and `nums[1] + nums[4] = -5 < target`

Java

Auto

```
1 class Solution {
2     public int countPairs(List<Integer> nums, int target) {
3         int count = 0;
4
5         for (int i = 0; i < nums.size(); i++) {
6             for (int j = i + 1; j < nums.size(); j++) {
7                 if (nums.get(i) + nums.get(j) < target) {
8                     count++;
9                 }
10            }
11        }
12    }
```

Saved

Ln 15, Col 2

Testcase

Test Result

Accepted

Runtime: 0 ms

Case 1

Case 2

Input

nums =
[-1,1,2,3,1]

target =
2

Output

3

2235. Add Two Integers

Solved

Easy

Topics

Companies

Given two integers `num1` and `num2`, return the **sum** of the two integers.

Example 1:

Input: `num1 = 12, num2 = 5`

Output: `17`

Explanation: `num1` is 12, `num2` is 5, and their sum is `12 + 5 = 17`, so 17 is returned.

Example 2:

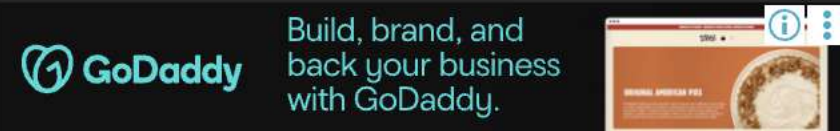
Input: `num1 = -10, num2 = 4`

Output: `-6`

Explanation: `num1 + num2 = -6`, so -6 is returned.

Constraints:

- `-100 <= num1, num2 <= 100`



Seen this question in a real interview before? 1/6

Yes

No

Code

Java Auto

Menu | Bookmarks | Syntax | Undo | Redo

```
1 class Solution {
2     public int sum(int num1, int num2) {
3         return num1+num2;
4     }
5 }
```

Saved

Ln 1, Col 1

Testcase | Test Result

Fullscreen | Up

Accepted Runtime: 0 ms

Case 1

Case 2

Input

num1 =
12

num2 =
5

Output

17

1323. Maximum 69 Number

Solved

Easy

Topics

Companies

Hint

You are given a positive integer `num` consisting only of digits `6` and `9`.

Return *the maximum number you can get by changing **at most** one digit* (`6` becomes `9`, and `9` becomes `6`).

Example 1:

Input: `num = 9669`

Output: `9969`

Explanation:

Changing the first digit results in 6669.

Changing the second digit results in 9969.

Changing the third digit results in 9699.

Changing the fourth digit results in 9666.

The maximum number is 9969.

Example 2:

Input: `num = 9996`

Output: `9999`

Explanation: Changing the last digit 6 to 9 results in the maximum number.

Example 3:

Input: `num = 9999`

Output: `9999`

Explanation: It is better not to apply any change.

Code

Java Auto

```

1  class Solution {
2      public int maximum69Number (int n) {
3          int diff=3,div=1,sum=0;
4          while(div<+n){
5              int ld=(n%(div*10))/div;
6              if(ld==6) sum=diff;
7              diff*=10;
8              div*=10;
9          }
10         return n+sum;
11     }
12 }
```

Saved

Ln 1, Col 1

☒ Testcase | Test Result

Accepted

Runtime: 0 ms

☒ Case 1

☒ Case 2

☒ Case 3

Input

num =

9669

Output

9969

Expected

9969

3099. Harshad Number

Solved

Easy | Topics | Companies | Hint

An integer divisible by the **sum** of its digits is said to be a **Harshad** number. You are given an integer `x`. Return *the sum of the digits* of `x` if `x` is a **Harshad** number, otherwise, return `-1`.

Example 1:

Input: `x = 18`

Output: `9`

Explanation:

The sum of digits of `x` is `9`. `18` is divisible by `9`. So `18` is a Harshad number and the answer is `9`.

Example 2:

Input: `x = 23`

Output: `-1`

Explanation:

The sum of digits of `x` is `5`. `23` is not divisible by `5`. So `23` is not a Harshad number and the answer is `-1`.

Constraints:

Code

Java | Auto

```
1 class Solution {
2     public int sumOfTheDigitsOfHarshadNumber(int x) {
3         int sum=0;
4         for (int n=x;n>0;n/=10)
5             sum +=n%10;
6         return (x %sum==0)? sum:-1;
7     }
8 }
```

Saved

Ln 1, Col 1

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 Case 2

Input

x =
18

Output

9

Expected

9

2843. Count Symmetric Integers

Solved

Easy

Topics

Companies

Hint

You are given two positive integers `low` and `high`.

An integer `x` consisting of `2 * n` digits is **symmetric** if the sum of the first `n` digits of `x` is equal to the sum of the last `n` digits of `x`. Numbers with an odd number of digits are never symmetric.

Return the **number of symmetric integers in the range** `[low, high]`.

Example 1:

Input: `low = 1, high = 100`

Output: 9

Explanation: There are 9 symmetric integers between 1 and 100: 11, 22, 33, 44, 55, 66, 77, 88, and 99.

Example 2:

Input: `low = 1200, high = 1230`

Output: 4

Explanation: There are 4 symmetric integers between 1200 and 1230: 1203, 1212, 1221, and 1230.

Constraints:

- `1 <= low <= high <= 104`

Code

Java | Auto

```
1 class Solution {
2     public int countSymmetricIntegers(int low, int high) {
3         int count = 0;
4
5         for (int i = low; i <= high; i++) {
6             String s = String.valueOf(i);
7
8             if (s.length() % 2 != 0) {
9                 continue;
10            }
11
12            int sum1 = 0, sum2 = 0;
```

Saved

Ln 27, Col 2

Testcase | Test Result

Accepted

Runtime: 0 ms

Case 1

Case 2

Input

low =

1

high =

100

Output

9

2119. A Number After a Double Reversal Solved ✓

[Easy](#) [Topics](#) [Companies](#) [Hint](#)

Reversing an integer means to reverse all its digits.

- For example, reversing `2021` gives `1202`. Reversing `12300` gives `321` as the **leading zeros are not retained**.

Given an integer `num`, **reverse** `num` to get `reversed1`, **then reverse** `reversed1` to get `reversed2`. Return `true` *if* `reversed2` *equals* `num`. Otherwise return `false`.

Example 1:

Input: `num = 526`

Output: `true`

Explanation: Reverse `num` to get `625`, then reverse `625` to get `526`, which equals `num`.

Example 2:

Input: `num = 1800`

Output: `false`

Explanation: Reverse `num` to get `81`, then reverse `81` to get `18`, which does not equal `num`.

Example 3:

Input: `num = 0`

Output: `true`

Explanation: Reverse `num` to get `0`, then reverse `0` to get `0`.

</> Code

Java ▾ 🔒 Auto

☰ 📖 {} ↶ ↷

```
1 class Solution {
2     public boolean isSameAfterReversals(int n) {
3         if(n%10!=0||n==0){
4             return true;
5         }
6         return false;
7     }
8 }
```

Saved

Ln 1, Col 1

☑ Testcase | >_ Test Result

⌂ ^

Accepted Runtime: 0 ms

☑ Case 1

☑ Case 2

☑ Case 3

Input

`num =`

`526`

Output

`true`

Expected

`true`

3736. Minimum Moves to Equal Array Elements III

Easy

Topics

Companies

Hint

You are given an integer array `nums`.

In one move, you may **increase** the value of any single element `nums[i]` by 1.

Return the **minimum total** number of **moves** required so that all elements in `nums` become **equal**.

Example 1:

Input: `nums = [2,1,3]`

Output: 3

Explanation:

To make all elements equal:

- Increase `nums[0] = 2` by 1 to make it 3.
- Increase `nums[1] = 1` by 1 to make it 2.
- Increase `nums[1] = 2` by 1 to make it 3.

Now, all elements of `nums` are equal to 3. The minimum total moves is 3.

Example 2:

Input: `nums = [4,4,5]`

Code

Java [Auto](#)

[≡](#) [🔖](#) [{ }](#) [↶](#) [↷](#)

```
1 class Solution {
2     public int minMoves(int[] nums) {
3         int max = 0;
4         int sum = 0;
5
6         for (int num : nums) {
7             if (num > max) {
8                 max = num;
9             }
10            sum += num;
11        }
12    }
```

Saved

Ln 15, Col 2

[✓ Testcase](#) | [> Test Result](#)

[⌂](#) [^](#)

Accepted Runtime: 0 ms

[✓ Case 1](#)

[✓ Case 2](#)

Input

nums =
[2,1,3]

Output

3

Expected

3

3550. Smallest Index With Digit Sum Equal to Index

Easy

Topics

Companies

Hint

You are given an integer array `nums`.

Return the **smallest** index `i` such that the sum of the digits of `nums[i]` is equal to `i`.

If no such index exists, return `-1`.

Example 1:

Input: `nums = [1,3,2]`

Output: 2

Explanation:

- For `nums[2] = 2`, the sum of digits is 2, which is equal to index `i = 2`. Thus, the output is 2.

Example 2:

Input: `nums = [1,10,11]`

Output: 1

Explanation:

- For `nums[1] = 10`, the sum of digits is `1 + 0 = 1`, which is equal to index `i = 1`.

Code

Java ▾ Auto

≡ |
 📖 |
 {} |
 ↶ |
 ↷

```

1  class Solution {
2      public int smallestIndex(int[] nums) {
3          for (int i = 0; i < nums.length; i++) {
4              int sum = 0;
5              int num = nums[i];
6
7              while (num > 0) {
8                  sum += num % 10;
9                  num /= 10;
10             }
11
12             if (sum == i) {

```

Saved

Ln 19, Col 2

☒ Testcase |
 >_ Test Result

[] |
 ^

Accepted Runtime: 0 ms

☒ Case 1

☒ Case 2

☒ Case 3

Input

nums =
 [1,3,2]

Output

2

Expected

2

3370. Smallest Number With All Set Bits

Easy | Topics | Companies | Hint

You are given a *positive* number `n`.

Return the **smallest** number `x` **greater than or equal to** `n`, such that the binary representation of `x` contains only **set bits**

Example 1:

Input: `n = 5`

Output: `7`

Explanation:

The binary representation of 7 is `"111"`.

Example 2:

Input: `n = 10`

Output: `15`

Explanation:

The binary representation of 15 is `"1111"`.

Example 3:

Input: `n = 3`

Output: `3`

Code

Java | Auto

```
1 class Solution {
2     public int smallestNumber(int n) {
3         int x = 1;
4
5         while (x - 1 < n) {
6             x <<= 1;
7         }
8
9         return x - 1;
10    }
11 }
```

Saved

Ln 11, Col 2

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input

n =
5

Output

7

Expected

7

2169. Count Operations to Obtain Zero

Easy | Topics | Companies | Hint

You are given two **non-negative** integers `num1` and `num2`.

In one **operation**, if `num1 >= num2`, you must subtract `num2` from `num1`, otherwise subtract `num1` from `num2`.

- For example, if `num1 = 5` and `num2 = 4`, subtract `num2` from `num1`, thus obtaining `num1 = 1` and `num2 = 4`. However, if `num1 = 4` and `num2 = 5`, after one operation, `num1 = 4` and `num2 = 1`.

Return the **number of operations** required to make either `num1 = 0` or `num2 = 0`.

Example 1:

Input: `num1 = 2, num2 = 3`

Output: 3

Explanation:

- Operation 1: `num1 = 2, num2 = 3`. Since `num1 < num2`, we subtract `num1` from `num2` and get `num1 = 2, num2 = 3 - 2 = 1`.
- Operation 2: `num1 = 2, num2 = 1`. Since `num1 > num2`, we subtract `num2` from `num1`.
- Operation 3: `num1 = 1, num2 = 1`. Since `num1 == num2`, we subtract `num2` from `num1`.

Now `num1 = 0` and `num2 = 1`. Since `num1 == 0`, we do not need to perform any further operations.

So the total number of operations required is 3.

Example 2:

Code

Java | Auto

```
1 class Solution {
2     public int countOperations(int num1, int num2) {
3         int count = 0;
4
5         while (num1 != 0 && num2 != 0) {
6             if (num1 >= num2) {
7                 num1 -= num2;
8             } else {
9                 num2 -= num1;
10            }
11            count++;
12        }
13    }
```

Saved

Ln 16, Col 2

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 Case 2

Input

num1 =
2

num2 =
3

Output

3

2169. Count Operations to Obtain Zero Solved

Easy | Topics | Companies | Hint

You are given two **non-negative** integers `num1` and `num2`.

In one **operation**, if `num1 >= num2`, you must subtract `num2` from `num1`, otherwise subtract `num1` from `num2`.

- For example, if `num1 = 5` and `num2 = 4`, subtract `num2` from `num1`, thus obtaining `num1 = 1` and `num2 = 4`. However, if `num1 = 4` and `num2 = 5`, after one operation, `num1 = 4` and `num2 = 1`.

Return the **number of operations** required to make either `num1 = 0` or `num2 = 0`.

Example 1:

Input: `num1 = 2, num2 = 3`
Output: 3
Explanation:

- Operation 1: `num1 = 2, num2 = 3`. Since `num1 < num2`, we subtract `num1` from `num2` and get `num1 = 2, num2 = 3 - 2 = 1`.
- Operation 2: `num1 = 2, num2 = 1`. Since `num1 > num2`, we subtract `num2` from `num1`.
- Operation 3: `num1 = 1, num2 = 1`. Since `num1 == num2`, we subtract `num2` from `num1`.

Now `num1 = 0` and `num2 = 1`. Since `num1 == 0`, we do not need to perform any further operations.
So the total number of operations required is 3.

Example 2:

```
Code
Java Auto
1 class Solution {
2     public int countOperations(int num1, int num2) {
3         int count = 0;
4
5         while (num1 != 0 && num2 != 0) {
6             if (num1 >= num2) {
7                 num1 -= num2;
8             } else {
9                 num2 -= num1;
10            }
11            count++;
12        }
13    }
14 }
```

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 Case 2

Input

num1 = 2

num2 = 3

Output

3

3750. Minimum Number of Flips to Reverse Binary String

Easy

Topics

Companies

Hint

You are given a **positive** integer `n`.

Let `s` be the **binary representation** of `n` without leading zeros.

The **reverse** of a binary string `s` is obtained by writing the characters of `s` in the opposite order.

You may flip any bit in `s` (change `0 → 1` or `1 → 0`). Each flip affects **exactly** one bit.

Return the **minimum** number of flips required to make `s` equal to the reverse of its original form.

Example 1:

Input: n = 7

Output: 0

Explanation:

The binary representation of 7 is "111". Its reverse is also "111", which is the same. Hence, no flips are needed.

Example 2:

Input: n = 10

Code

Java Auto

```
1 class Solution {
2     public int minimumFlips(int n) {
3         String s = Integer.toBinaryString(n);
4         int count = 0;
5
6         for (int i = 0; i < s.length() / 2; i++) {
7             if (s.charAt(i) != s.charAt(s.length() - 1 - i)) {
8                 count += 2;
9             }
10        }
11
12        return count;
13    }
14}
```

SavedLn 14, Col 2

Testcase Test Result

AcceptedRuntime: 0 ms

Case 1

Case 2

Input

n = 7

Output

0

Expected

0

3079. Find the Sum of Encrypted Integers

Easy

Topics

Companies

Hint

You are given an integer array `nums` containing **positive** integers. We define a function `encrypt` such that `encrypt(x)` replaces **every** digit in `x` with the **largest** digit in `x`. For example, `encrypt(523) = 555` and `encrypt(213) = 333`.

Return *the sum of encrypted elements*.

Example 1:

Input: `nums = [1,2,3]`

Output: 6

Explanation: The encrypted elements are `[1,2,3]`. The sum of encrypted elements is `1 + 2 + 3 == 6`.

Example 2:

Input: `nums = [10,21,31]`

Output: 66

Explanation: The encrypted elements are `[11,22,33]`. The sum of encrypted elements is `11 + 22 + 33 == 66`.

Constraints:

- `1 <= nums.length <= 50`

Java

Auto

```
1 class Solution {
2     public int sumOfEncryptedInt(int[] nums) {
3         int total = 0;
4
5         for (int num : nums) {
6             int maxDigit = 0;
7             int temp = num;
8             int len = 0;
9
10            while (temp > 0) {
11                int digit = temp % 10;
12                maxDigit = Math.max(maxDigit, digit);
```

Accepted

Runtime: 0 ms

☒ Case 1

☒ Case 2

Input

nums =

[1,2,3]

Output

6

Expected

6

2946. Matrix Similarity After Cyclic Shifts

Easy

Topics

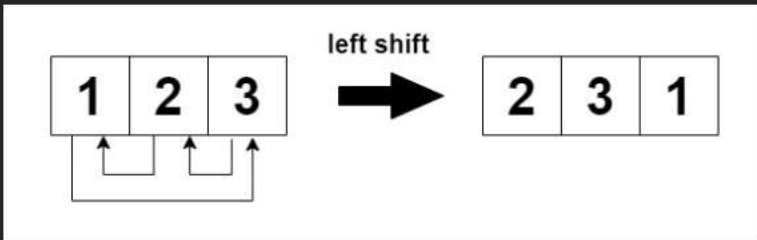
Companies

Hint

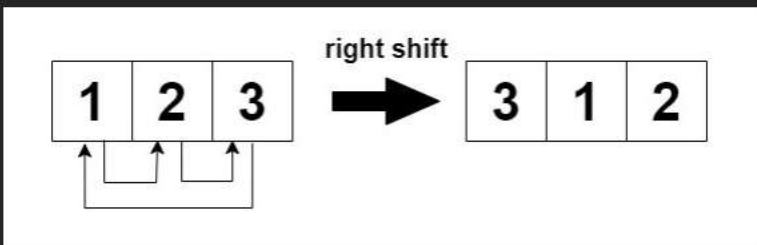
You are given an $m \times n$ integer matrix `mat` and an integer `k`. The matrix rows are 0-indexed.

The following proccess happens `k` times:

- **Even-indexed** rows (0, 2, 4, ...) are cyclically shifted to the left.



- **Odd-indexed** rows (1, 3, 5, ...) are cyclically shifted to the right.



Return `true` if the final modified matrix after `k` steps is identical to the original matrix, and `false` otherwise.

Example 1:

Input: `mat = [[1,2,3],[4,5,6],[7,8,9]]`, `k = 4`

Output: `false`

Explanation:

Java

Auto

```
1 class Solution {
2     public boolean areSimilar(int[][] mat, int k) {
3         int n = mat.length;
4         int m = mat[0].length;
5
6         k = k % m;
7
8         for (int i = 0; i < n; i++) {
9             int[] shifted = new int[m];
10
11             for (int j = 0; j < m; j++) {
12                 shifted[(j + k) % m] = mat[i][j];
```

Saved

Ln 24, Col 2

Testcase

Test Result

Accepted

Runtime: 0 ms

Case 1

Case 2

Case 3

Input

mat =
[[1,2,3],[4,5,6],[7,8,9]]

k =
4

Output

false

509. Fibonacci Number

Easy Topics Companies

The **Fibonacci numbers**, commonly denoted $F(n)$ form a sequence, called the **Fibonacci sequence**, such that each number is the sum of the two preceding ones, starting from 0 and 1. That is,

$$F(0) = 0, F(1) = 1$$
$$F(n) = F(n - 1) + F(n - 2), \text{ for } n > 1.$$

Given n , calculate $F(n)$.

Example 1:

Input: $n = 2$
Output: 1
Explanation: $F(2) = F(1) + F(0) = 1 + 0 = 1.$

Example 2:

Input: $n = 3$
Output: 2
Explanation: $F(3) = F(2) + F(1) = 1 + 1 = 2.$

Example 3:

Input: $n = 4$
Output: 3
Explanation: $F(4) = F(3) + F(2) = 2 + 1 = 3.$

Constraints:

Code

Java Auto

```
1 class Solution {
2     public int fib(int n) {
3         if (n <= 1) {
4             return n;
5         }
6
7         int a = 0, b = 1;
8
9         for (int i = 2; i <= n; i++) {
10             int c = a + b;
11             a = b;
12             b = c;
```

Saved

Ln 17, Col 2

Testcase Test Result

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input

n =
2

Output

1

Expected

1

2180. Count Integers With Even Digit Sum

Solved ✓

Easy

Topics

Companies

Hint

Given a positive integer `num`, return *the number of positive integers **less than or equal to** `num` whose digit sums are **even**.*

The **digit sum** of a positive integer is the sum of all its digits.

Example 1:

Input: `num = 4`

Output: 2

Explanation:

The only integers less than or equal to 4 whose digit sums are even are 2 and 4.

Example 2:

Input: `num = 30`

Output: 14

Explanation:

The 14 integers less than or equal to 30 whose digit sums are even are 2, 4, 6, 8, 11, 13, 15, 17, 19, 20, 22, 24, 26, and 28.

Constraints:

- `1 <= num <= 1000`

Code

Java Auto

```
1 class Solution {
2     public int countEven(int num) {
3         int temp = num;
4         int digitSum = 0;
5         while (temp > 0) {
6             digitSum += temp % 10;
7             temp /= 10;
8         }
9         return digitSum % 2 == 0 ? num / 2 : (num - 1) / 2;
10    }
11 }
```

Saved

Ln 1, Col 1

Testcase Test Result

Accepted Runtime: 0 ms

Case 1

Case 2

Input

num =
4

Output

2

Expected

2

3622. Check Divisibility by Digit Sum and Product

Solved

Easy

Topics

Companies

Hint

You are given a positive integer n . Determine whether n is divisible by the **sum** of the following two values:

- The **digit sum** of n (the sum of its digits).
- The **digit product** of n (the product of its digits).

Return `true` if n is divisible by this sum; otherwise, return `false`.

Example 1:

Input: $n = 99$

Output: `true`

Explanation:

Since 99 is divisible by the sum ($9 + 9 = 18$) plus product ($9 * 9 = 81$) of its digits (total 99), the output is true.

Example 2:

Input: $n = 23$

Output: `false`

Explanation:

Code

Java Auto

```
1 class Solution {
2     public boolean checkDivisibility(int num) {
3         int sum = 0;
4         int product = 1;
5         int temp = num;
6
7         while (temp > 0) {
8             int digit = temp % 10;
9             sum += digit;
10            product *= digit;
11            temp /= 10;
12        }
```

Saved

Ln 16, Col 2

Testcase | Test Result

Accepted

Runtime: 0 ms

Case 1

Case 2

Input

$n =$

99

Output

true

Expected

true

258. Add Digits

Solved

Easy

Topics

Companies

Hint

Given an integer `num`, repeatedly add all its digits until the result has only one digit, and return it.

Example 1:

Input: `num = 38`
Output: `2`
Explanation: The process is
`38 --> 3 + 8 --> 11`
`11 --> 1 + 1 --> 2`
Since 2 has only one digit, return it.

Example 2:

Input: `num = 0`
Output: `0`

Constraints:

- $0 \leq num \leq 2^{31} - 1$

Follow up: Could you do it without any loop/recursion in $O(1)$ runtime?

Code

Java Auto

```
1 class Solution {
2     public int addDigits(int num) {
3         while (num >= 10) {
4             int sum = 0;
5
6             while (num > 0) {
7                 sum += num % 10;
8                 num /= 10;
9             }
10
11             num = sum;
12         }
13     }
14 }
```

SavedLn 16, Col 2

TestcaseTest Result

AcceptedRuntime: 0 ms

Case 1Case 2

Input

num = 38

Output

2

Expected

2